Task-1

1.Problem Statement

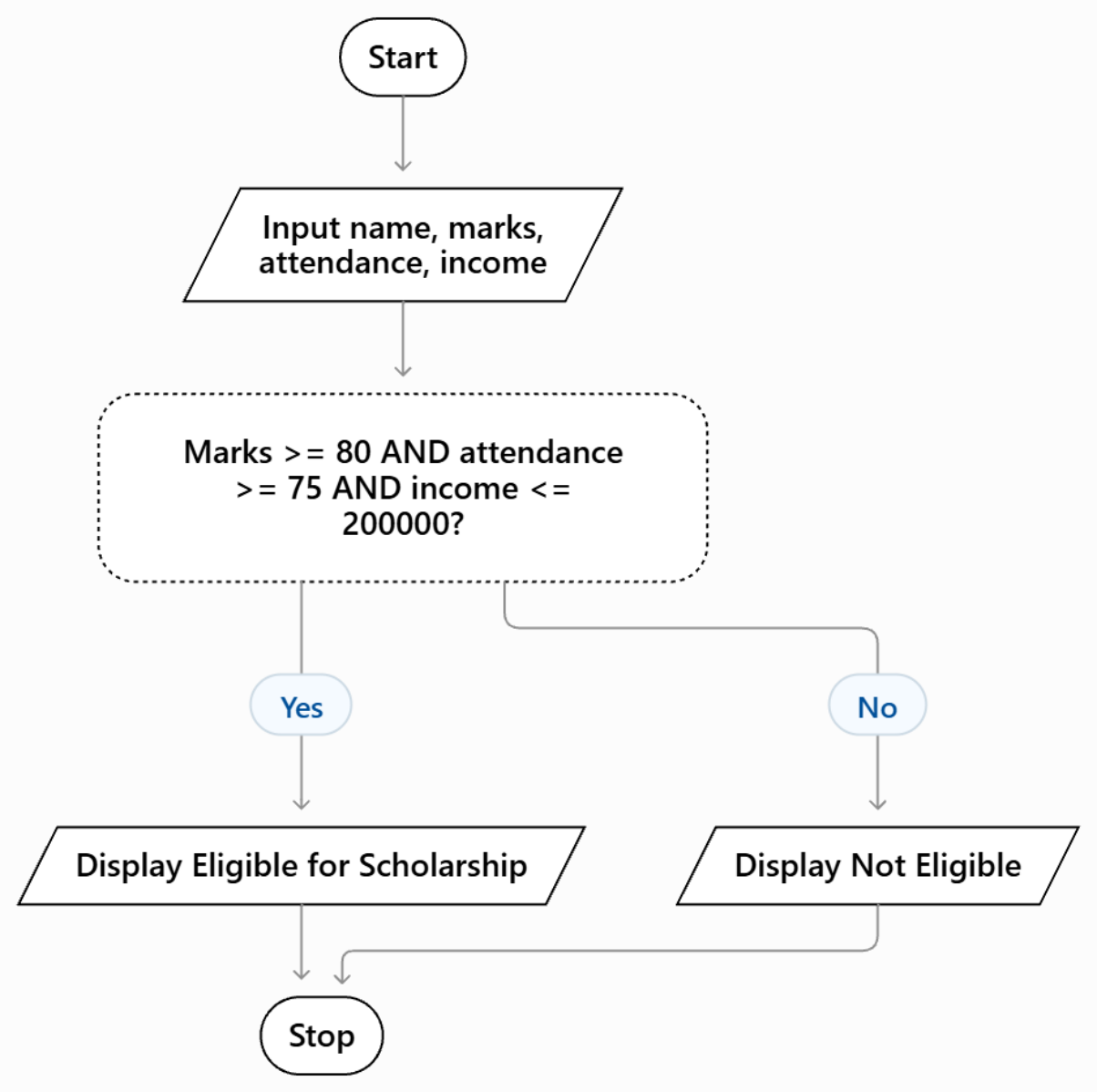
Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2.Algorithm

1. Start.
2. Read student name.
3. Read aggregate marks.
4. Read attendance percentage.
5. Read family income.
6. Check whether marks are greater than or equal to 80, attendance is greater than or equal to 75, and income is less than or equal to 2,00,000.
7. If all conditions are true, display "Eligible for Scholarship".
8. Otherwise, display "Not Eligible".
9. Stop.

BEGIN

3.flowchart



4.Pseudocode

BEGIN

INPUT student name

INPUT aggregate marks

INPUT attendance

INPUT family income

IF marks >= 80 AND attendance >= 75

AND income <= 200000 THEN

PRINT "Eligible for Scholarship"

ELSE

PRINT "Not Eligible"

END IF

STOP

5.Python Program

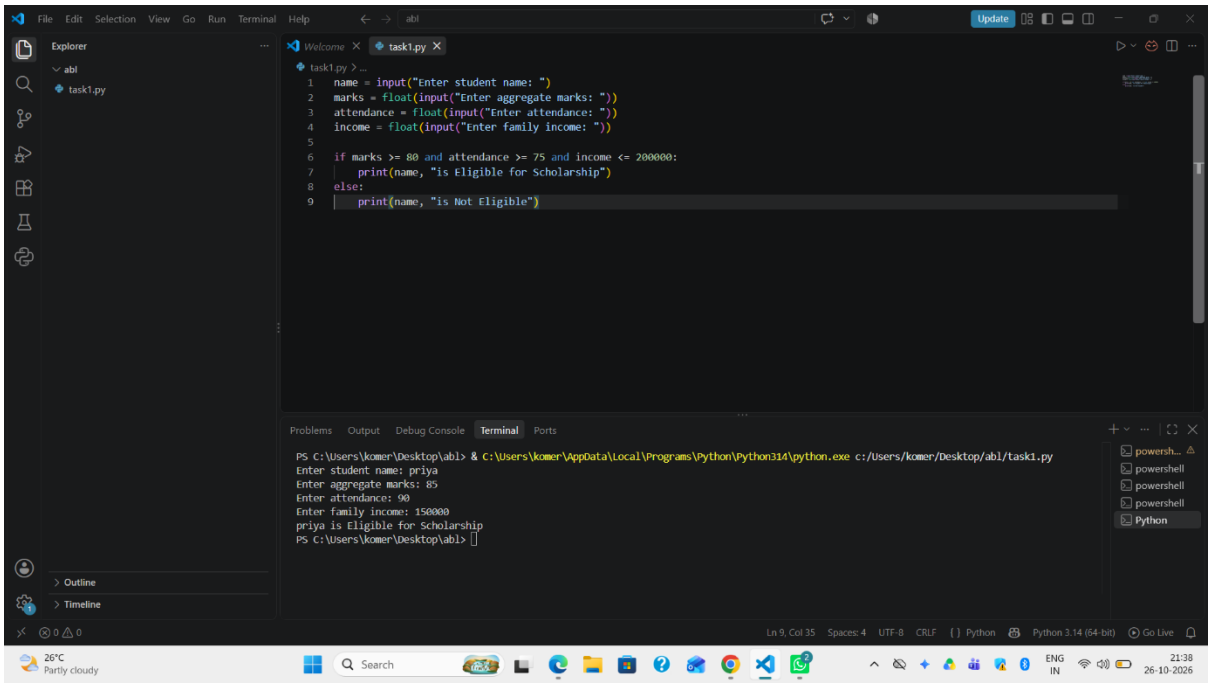
```
name = input("Enter student name: ")
marks = float(input("Enter aggregate marks: "))
attendance = float(input("Enter attendance: "))
income = float(input("Enter family income: "))

if marks >= 80 and attendance >= 75 and income <= 200000:
  print(name, "is Eligible for Scholarship")
else:
  print(name, "is Not Eligible")
```

6.Test Cases

Test	Marks	Attendance	Income	Expected Output
1	85	90	150000	Eligible
2	70	90	150000	Not Eligible
3	85	60	150000	Not Eligible
4	85	90	250000	Not Eligible
5	80	75	200000	Eligible

7.Output Screenshot



8.Conclusion

The Python program successfully checks scholarship eligibility based on student marks, attendance, and family income. It uses conditional statements and logical operators to determine whether a student is eligible for a scholarship.

Task-2

1. Problem Statement

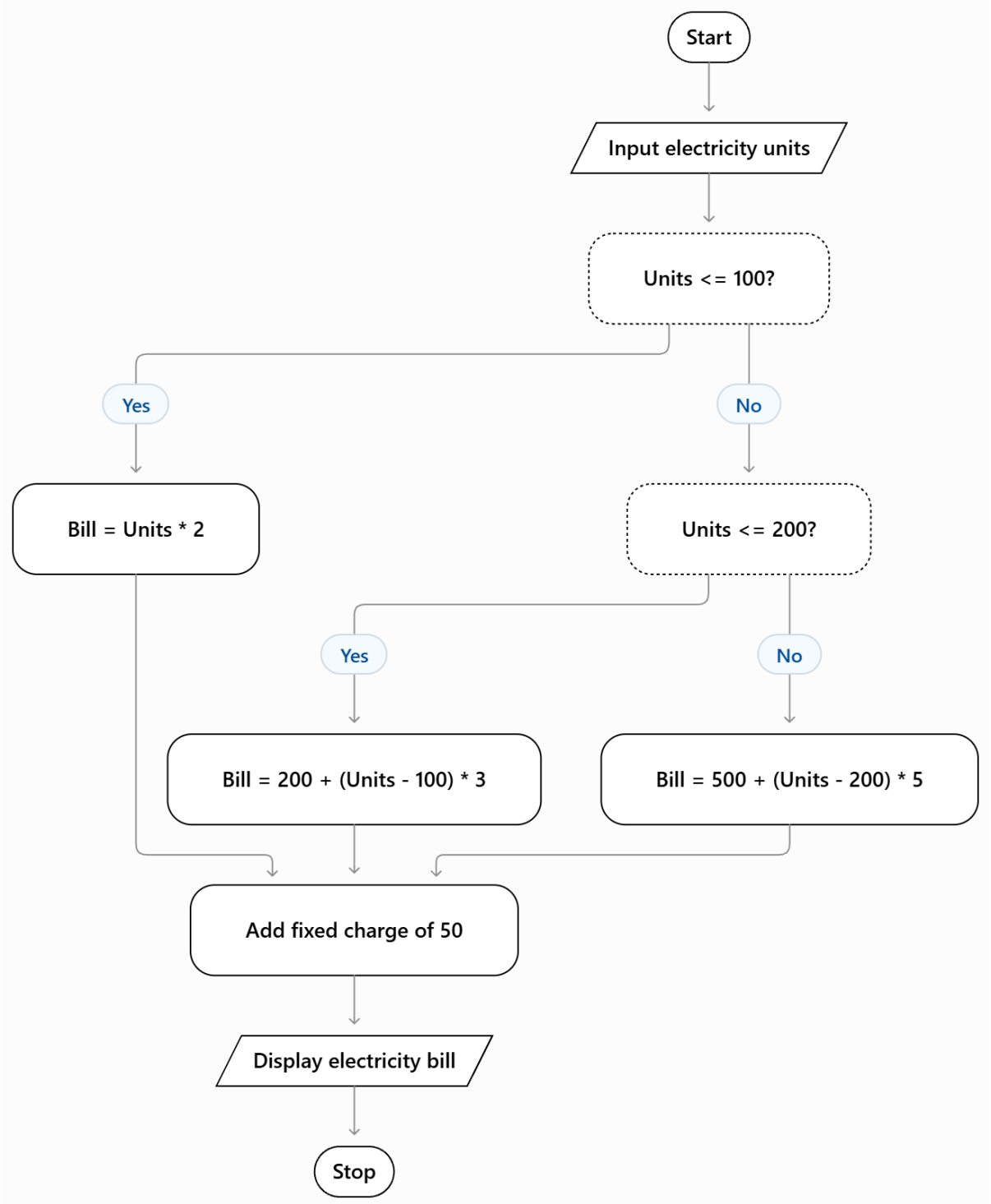
Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. Algorithm

1. Start.
2. Read the number of electricity units.
3. If units are less than or equal to 100, calculate bill as units × 2.

4. Else if units are less than or equal to 200, calculate the first 100 units at ₹2 and remaining units at ₹3.
5. Otherwise, calculate the first 100 units at ₹2, next 100 units at ₹3, and remaining units at ₹5.
6. Add ₹50 as a fixed charge.
7. Display the total electricity bill.
8. Stop.

3.Flowchart



4. Pseudocode

BEGIN

INPUT units

IF units <= 100 THEN

  bill = units * 2

ELSE IF units <= 200 THEN

  bill = 100 * 2 + (units - 100) * 3

ELSE

  bill = 100 * 2 + 100 * 3 + (units - 200) * 5

END IF

bill = bill + 50

PRINT electricity bill

STOP

5.Python Program (Complete Code)

```
units = int(input("Enter electricity units: "))

if units <= 100:
  bill = units * 2
elif units <= 200:
  bill = 100 * 2 + (units - 100) * 3
else:
  bill = 100 * 2 + 100 * 3 + (units - 200) * 5

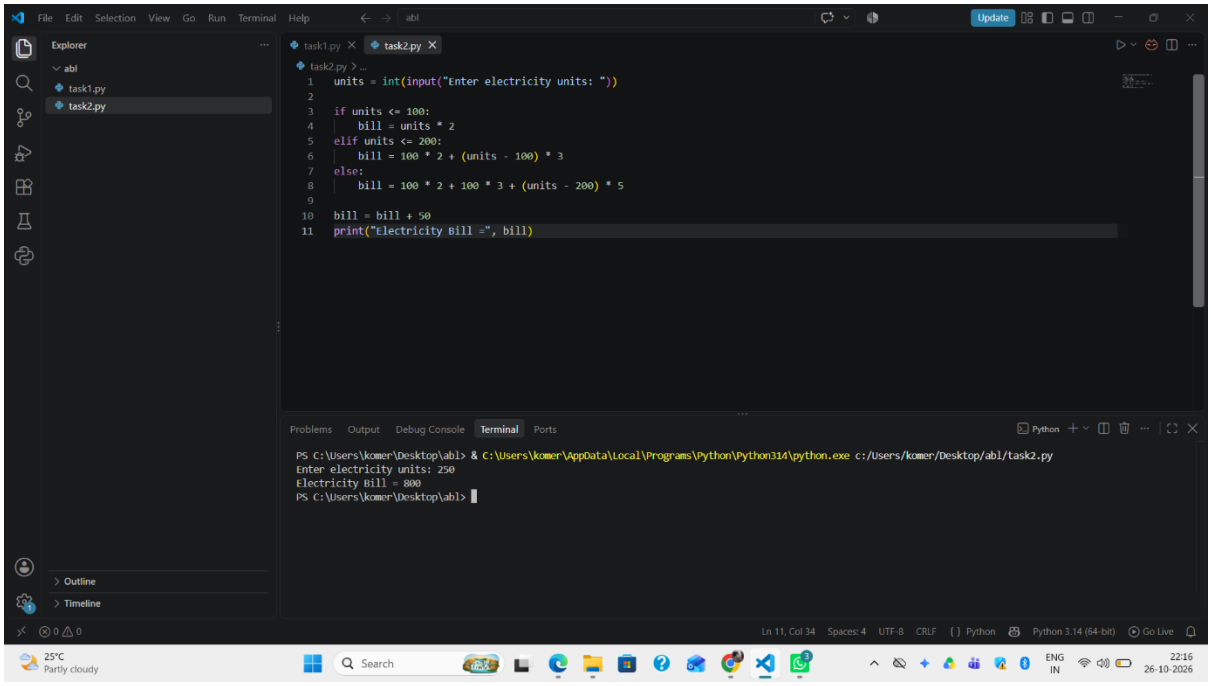
bill = bill + 50
print("Electricity Bill =", bill)
```

6. Test Cases

Test	Units	Calculation	Expected Bill
1	50	50 × 2 + 50	₹150

2	100	100 × 2 + 50	₹250
3	150	200 + 150 + 50	₹400
4	200	200 + 300 + 50	₹550
5	250	200 + 300 + 250 + 50	₹800

7. Output Screenshot



8. Conclusion

The Python program successfully calculates the electricity bill based on the units consumed by the customer. It uses if, elif, and else conditional statements to apply different electricity charges and adds a fixed charge of ₹50 to calculate the final bill.

Your second case study is ready! Next, we can prepare the third program in the same 10-section format.

Task-3

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Algorithm

Start.

Read consultation charges.

Read diagnostic test charges.

Read treatment charges.

Calculate the total bill by adding all charges.

Check whether the total bill is greater than or equal to ₹5,000.

If the condition is true, calculate a 10% discount.

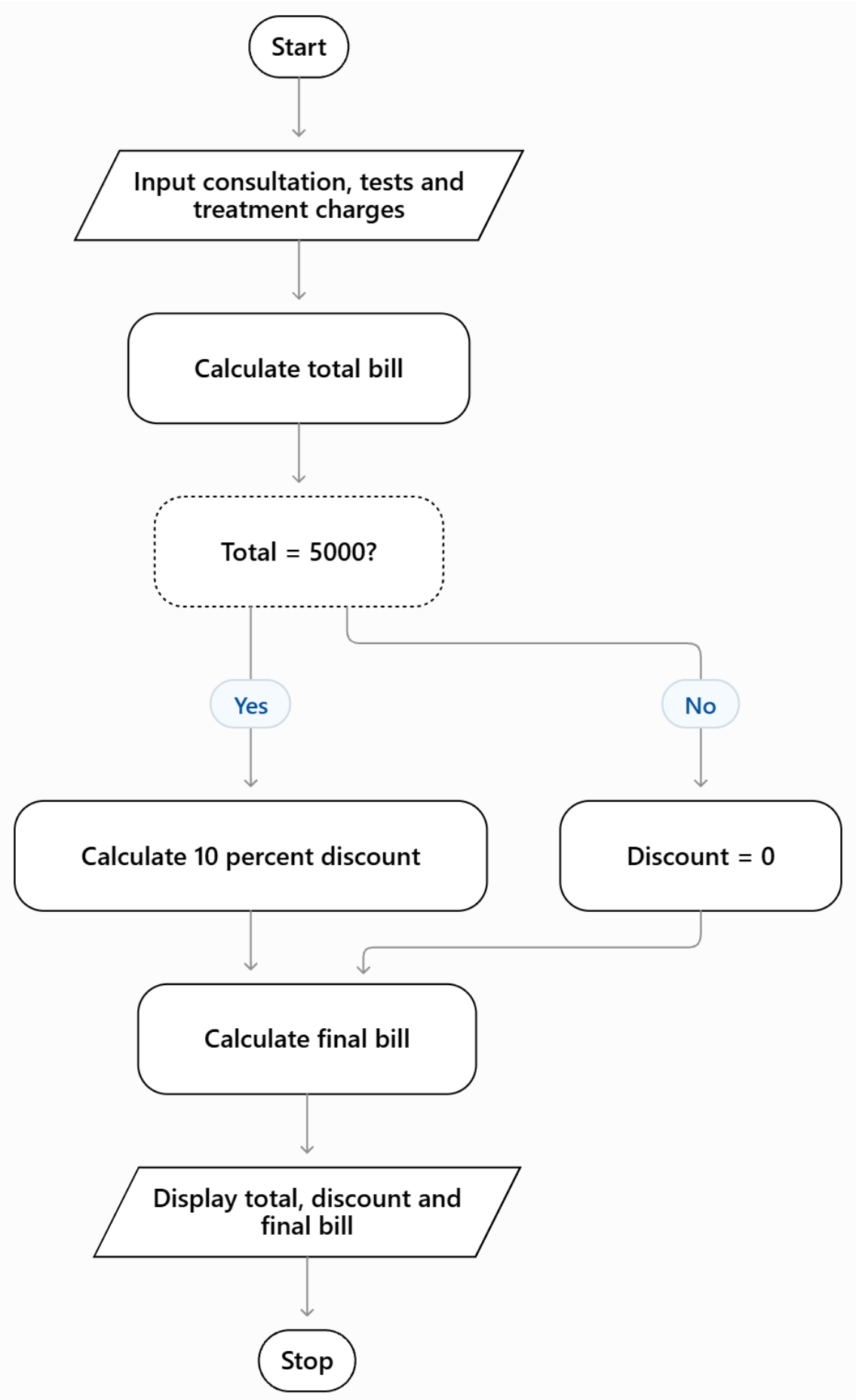
Otherwise, set the discount to zero.

Subtract the discount from the total bill.

Display total bill, discount, and final bill.

Stop.

3.flowchart


Start
Input consultation, tests and treatment charges
Calculate total bill
Total = 5000?
Yes
No
Calculate 10 percent discount
Discount = 0
Calculate final bill
Display total, discount and final bill
Stop

4. Pseudocode

BEGIN

INPUT consultation

INPUT tests

INPUT treatment

total = consultation + tests + treatment

IF total >= 5000 THEN

  discount = total * 0.10

ELSE

  discount = 0

END IF

final_bill = total - discount

PRINT total

PRINT discount

PRINT final_bill

STOP

5.Python Program (Complete Code)

```
total = consultation + tests + treatment

if total >= 5000:
  discount = total * 0.10
else:
```

```
    discount = 0

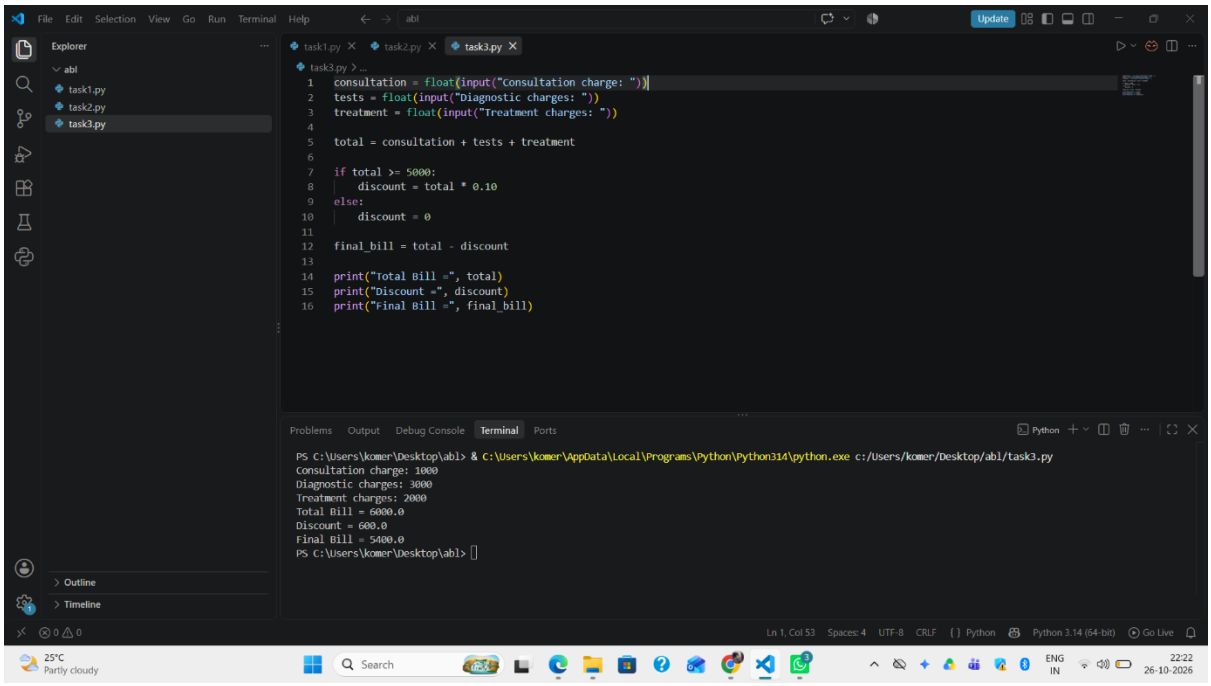
final_bill = total - discount

print("Total Bill =", total)
print("Discount =", discount)
print("Final Bill =", final_bill)
```

6. Test Cases

Test	Consultation	Tests	Treatment	Expected Final Bill
1	1000	1000	2000	₹4000
2	2000	1500	2000	₹4950
3	1000	2000	3000	₹5400
4	500	500	1000	₹2000
5	2000	2000	1000	₹4500

7. Output Screenshot



8. Conclusion

The Python program successfully calculates the hospital bill and applies a 10% discount when the total amount is ₹5,000 or more. It uses arithmetic operations and conditional statements to calculate and display the final bill amount.

Task-4

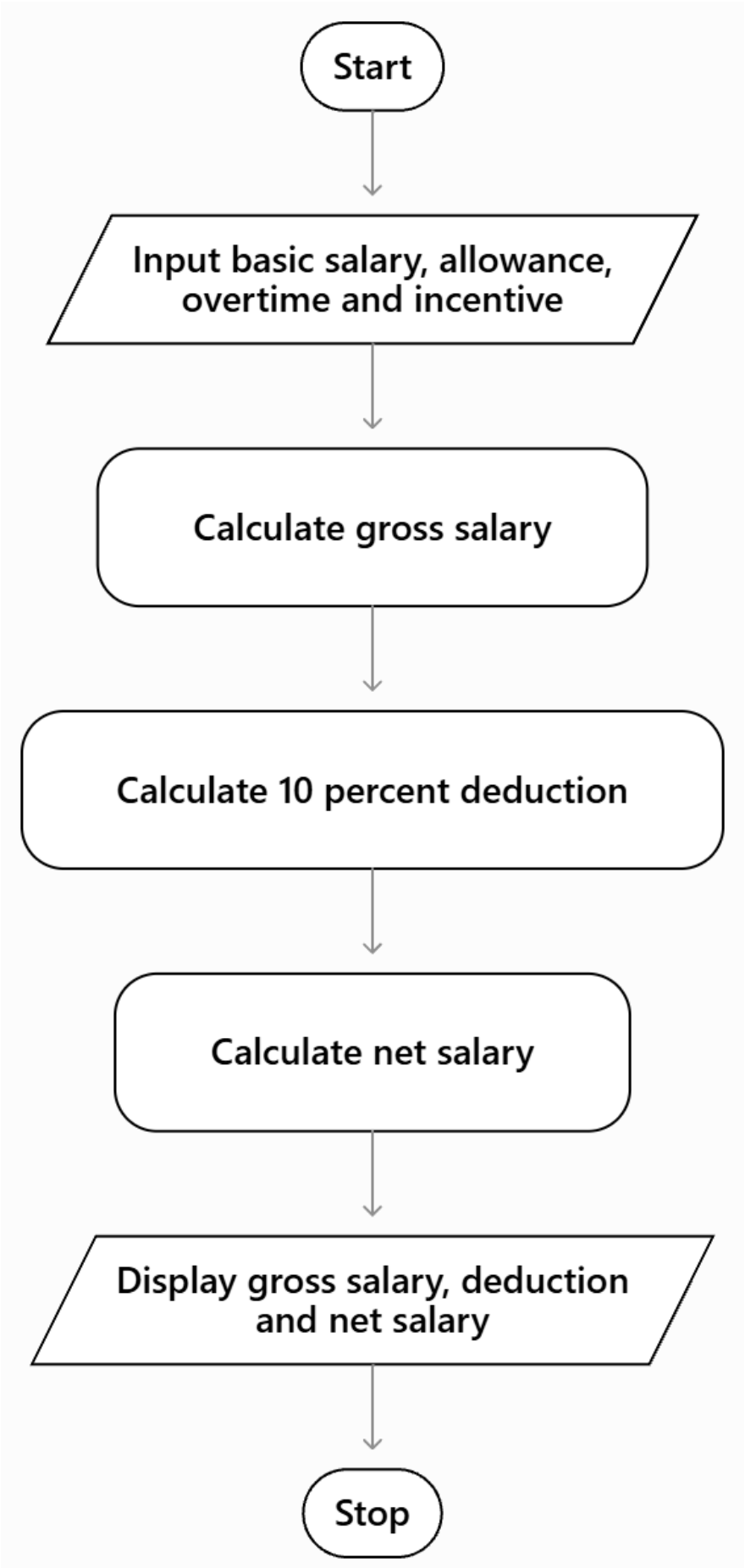
1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Algorithm

1. Start.
2. Read basic salary.
3. Read allowances.
4. Read overtime pay.
5. Read incentives.
6. Calculate gross salary by adding all four amounts.
7. Calculate deduction as 10% of gross salary.
8. Calculate net salary by subtracting deduction from gross salary.
9. Display gross salary, deduction, and net salary.
10. Stop.

3.Flowchart


Start
Input basic salary, allowance, overtime and incentive
Calculate gross salary
Calculate 10 percent deduction
Calculate net salary
Display gross salary, deduction and net salary
Stop

4.Pseudocode

BEGIN

INPUT basic

INPUT allowance

INPUT overtime

INPUT incentive

gross = basic + allowance + overtime + incentive

deduction = gross * 0.10

net_salary = gross - deduction

PRINT gross

PRINT deduction

PRINT net_salary

STOP

5.Python Program

```
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowances: "))
overtime = float(input("Enter overtime pay: "))
incentive = float(input("Enter incentives: "))

gross = basic + allowance + overtime + incentive
deduction = gross * 0.10
net_salary = gross - deduction

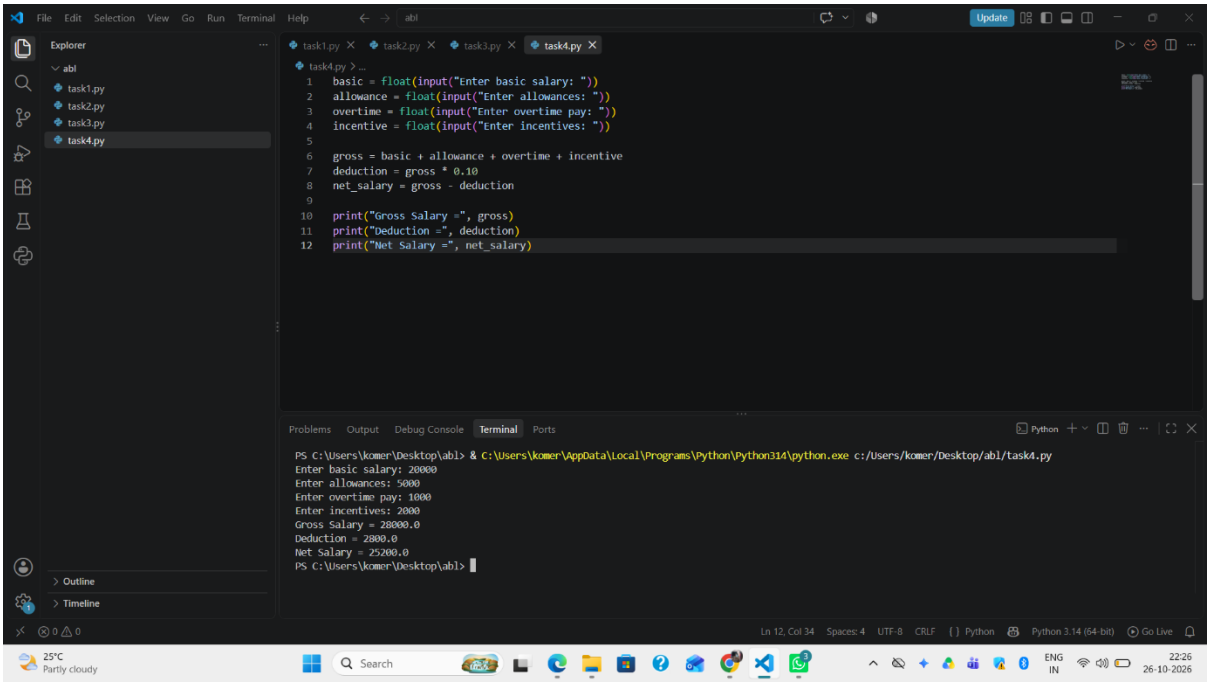
print("Gross Salary =", gross)
```

```
print("Deduction =", deduction)
print("Net Salary =", net_salary)
```

6. Test Cases

Test	Basic	Allowance	Overtime	Incentive	Net Salary
1	20000	3000	1000	1000	22500
2	15000	2000	500	500	16200
3	30000	5000	2000	3000	36000
4	10000	1000	500	500	10800
5	25000	4000	1000	2000	28800

7.Output Screenshot



8. Conclusion

The Python program successfully calculates the employee's gross salary, deduction, and net salary using arithmetic operations. It applies a 10% deduction to the gross salary and displays the final net salary.

Task-5

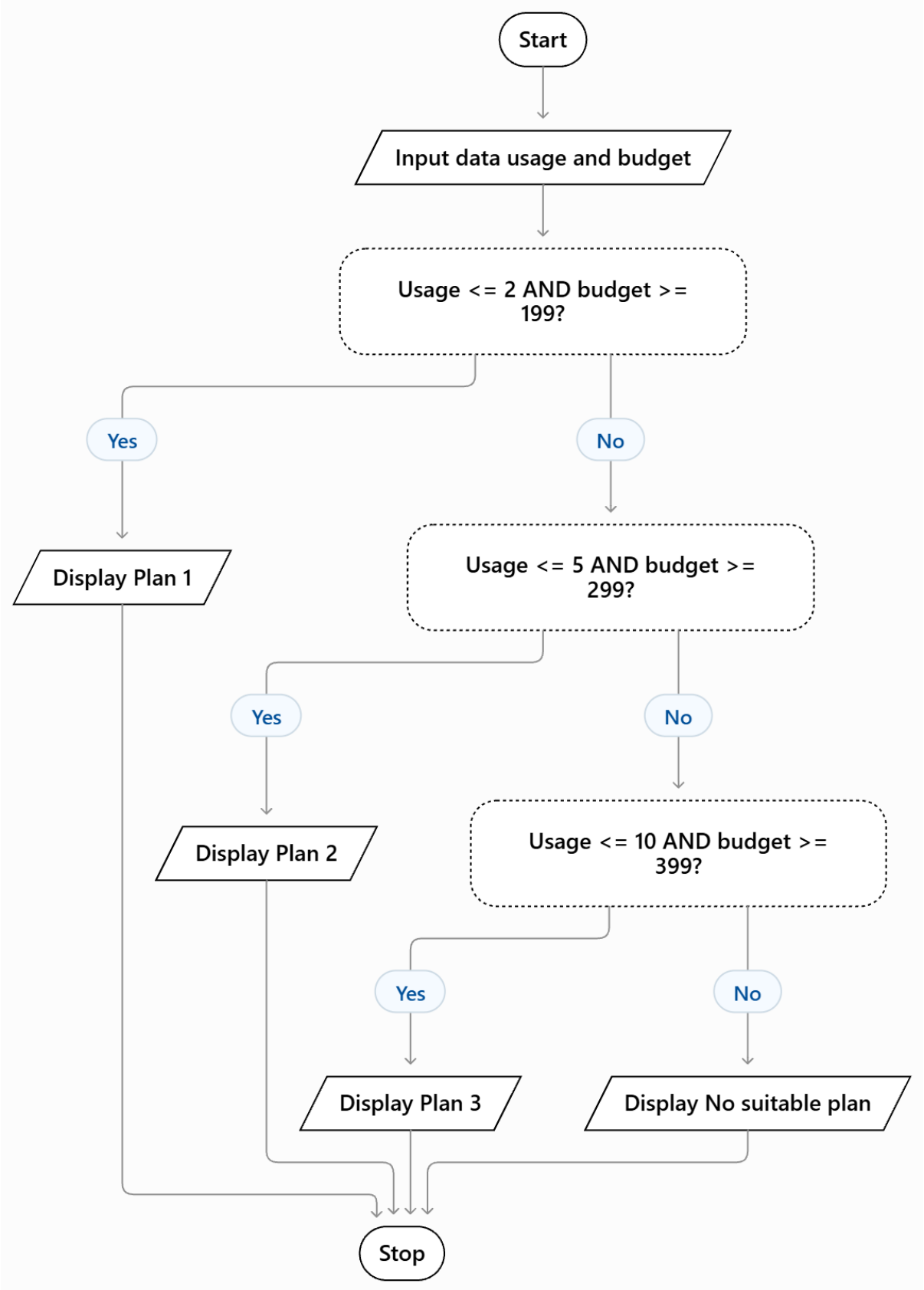
1. Problem Statement

Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan

2. Algorithm

1. Start.
2. Read data usage in GB.
3. Read the user's budget.
4. Assign prices to three mobile data plans.
5. Check whether usage is less than or equal to 2 GB and budget is at least ₹199.
6. If true, display Plan 1.
7. Otherwise, check whether usage is less than or equal to 5 GB and budget is at least ₹299.
8. If true, display Plan 2.
9. Otherwise, check whether usage is less than or equal to 10 GB and budget is at least ₹399.
10. If true, display Plan 3.
11. Otherwise, display "No suitable plan available".
12. Stop.

3.Flowchart


Start
Input data usage and budget
Usage <= 2 AND budget >= 199?
Yes
No
Display Plan 1
Usage <= 5 AND budget >= 299?
Yes
No
Display Plan 2
Usage <= 10 AND budget >= 399?
Yes
No
Display Plan 3
Display No suitable plan
Stop

4. Pseudocode

BEGIN

INPUT usage

INPUT budget

plan1 = 199

plan2 = 299

plan3 = 399

IF usage <= 2 AND plan1 <= budget THEN

  PRINT "Best Plan: Plan 1"

ELSE IF usage <= 5 AND plan2 <= budget THEN

  PRINT "Best Plan: Plan 2"

ELSE IF usage <= 10 AND plan3 <= budget THEN

  PRINT "Best Plan: Plan 3"

ELSE

  PRINT "No suitable plan available"

END IF

STOP

5.Python Program (Complete Code)

```
usage = float(input("Enter data usage in GB: "))
```

```
budget = float(input("Enter budget: "))

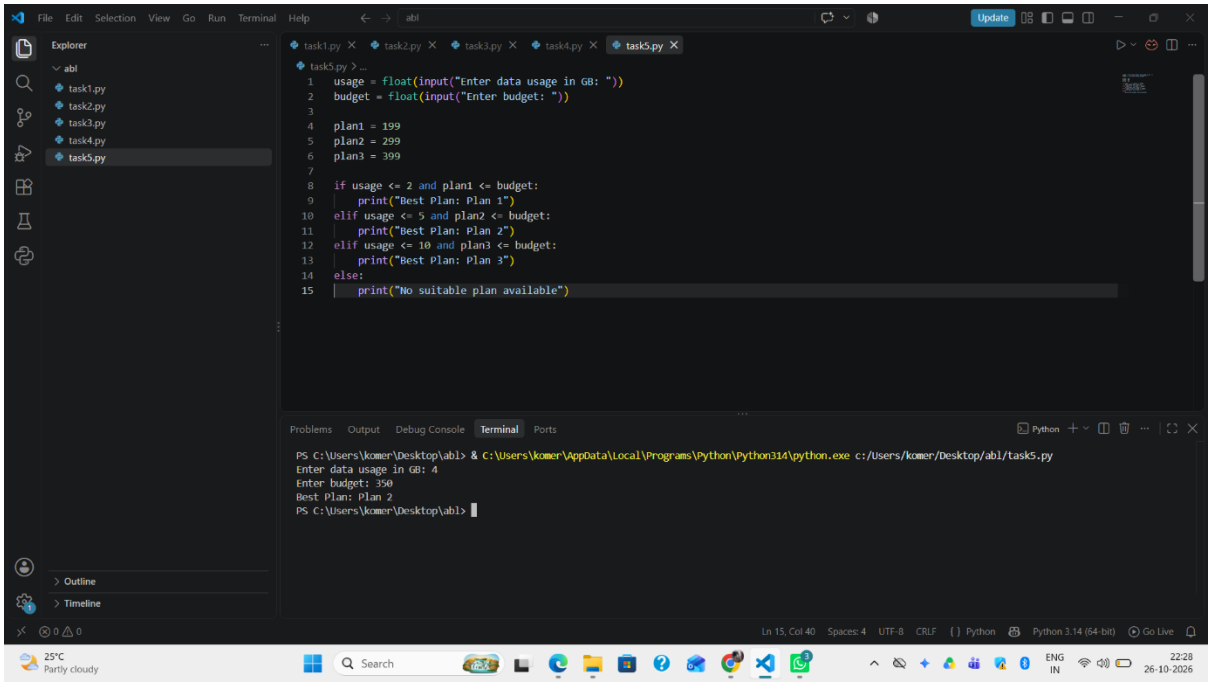
plan1 = 199
plan2 = 299
plan3 = 399

if usage <= 2 and plan1 <= budget:
   print("Best Plan: Plan 1")
elif usage <= 5 and plan2 <= budget:
   print("Best Plan: Plan 2")
elif usage <= 10 and plan3 <= budget:
   print("Best Plan: Plan 3")
else:
   print("No suitable plan available")
```

6. Test Cases

Test	Usage (GB)	Budget	Expected Output
1	1.5	₹250	Plan 1
2	4	₹350	Plan 2
3	8	₹450	Plan 3
4	3	₹200	No suitable plan
5	12	₹500	No suitable plan

7. Output Screenshot



8. Conclusion

The Python program successfully selects a suitable mobile data plan based on data usage and budget. It uses conditional statements and logical operators to compare different plans and display the best available option.

Task-6

1. Problem Statement

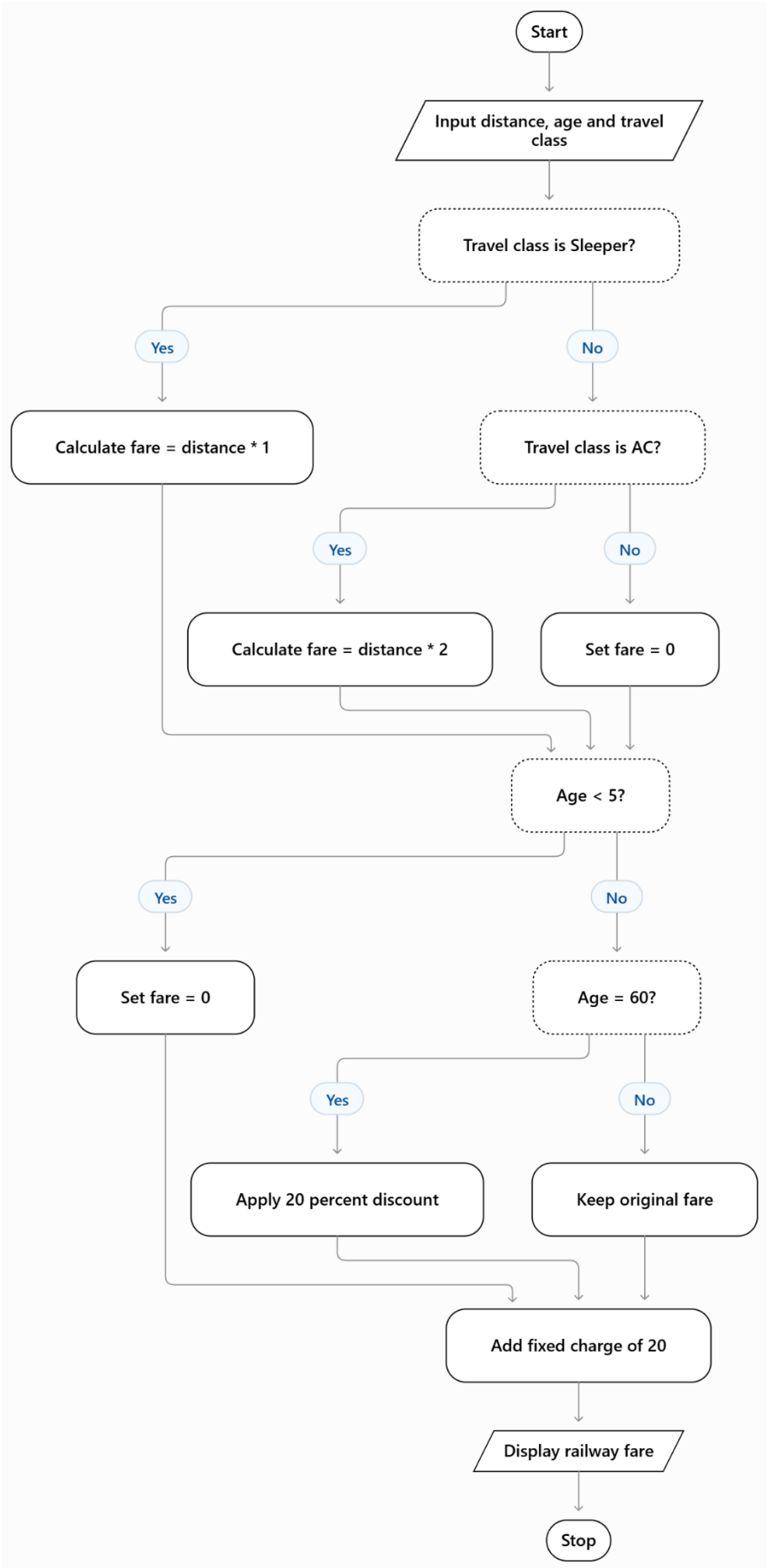
Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2. Algorithm

1. Start.
2. Read distance in kilometres.
3. Read passenger age.
4. Read travel class.
5. Convert travel class into lowercase.

6. If travel class is Sleeper, calculate fare as distance × 1.
7. Else if travel class is AC, calculate fare as distance × 2.
8. Otherwise, set fare to zero.
9. If passenger age is below 5, set fare to zero.
10. Else if passenger age is 60 or above, apply a 20% discount.
11. Add ₹20 to the fare.
12. Display the railway fare.
13. Stop.

3. Flowchart


Start
Input distance, age and travel class
Travel class is Sleeper?
Yes
No
Calculate fare = distance * 1
Travel class is AC?
Yes
No
Calculate fare = distance * 2
Set fare = 0
Age < 5?
Yes
No
Set fare = 0
Age = 60?
Yes
No
Apply 20 percent discount
Keep original fare
Add fixed charge of 20
Display railway fare
Stop

4.Pseudocode

```
BEGIN

INPUT distance

INPUT age

INPUT travel_class

CONVERT travel_class TO lowercase

IF travel_class == "sleeper" THEN
   fare = distance * 1
ELSE IF travel_class == "ac" THEN
   fare = distance * 2
ELSE
   fare = 0
END IF

IF age < 5 THEN
   fare = 0
ELSE IF age >= 60 THEN
   fare = fare * 0.8
END IF

fare = fare + 20

PRINT "Railway Fare =", fare
```

STOP

5.Python Program

```
distance = float(input("Enter distance in km: "))

age = int(input("Enter passenger age: "))

travel_class = input("Enter class (Sleeper/AC): ").lower()


if travel_class == "sleeper":

  fare = distance * 1

elif travel_class == "ac":

  fare = distance * 2

else:

  fare = 0


if age < 5:

  fare = 0

elif age >= 60:

  fare = fare * 0.8


fare = fare + 20

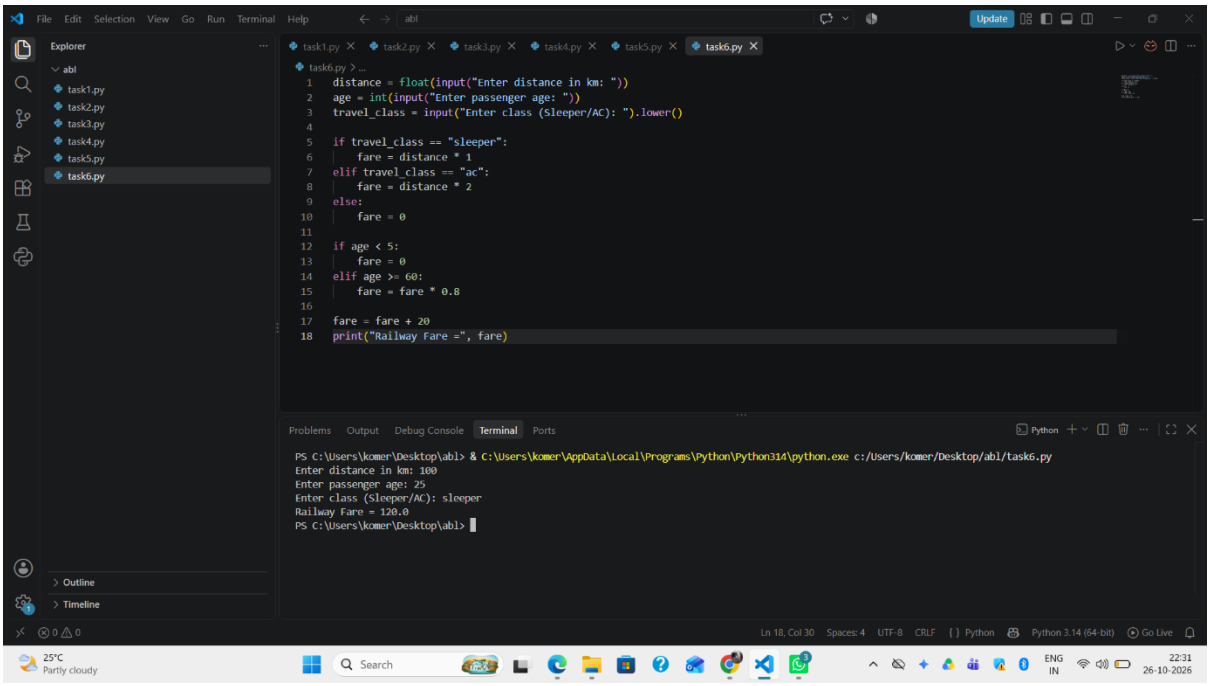
print("Railway Fare =", fare)
```

6. Test Cases

Test	Distance	Age	Class	Expected Fare
1	100 km	30	Sleeper	₹120
2	100 km	30	AC	₹220
3	100 km	65	AC	₹180
4	100 km	3	Sleeper	₹20

5	50 km	60	Sleeper ₹60

7.Output Screenshot



8. Conclusion

The Python program successfully calculates railway fare based on travel distance, passenger age, and travel class. It uses conditional statements to calculate different fares, provide discounts for senior citizens, and offer free base fare for children below 5 years. A fixed charge of ₹20 is added to the final fare.

Task-7

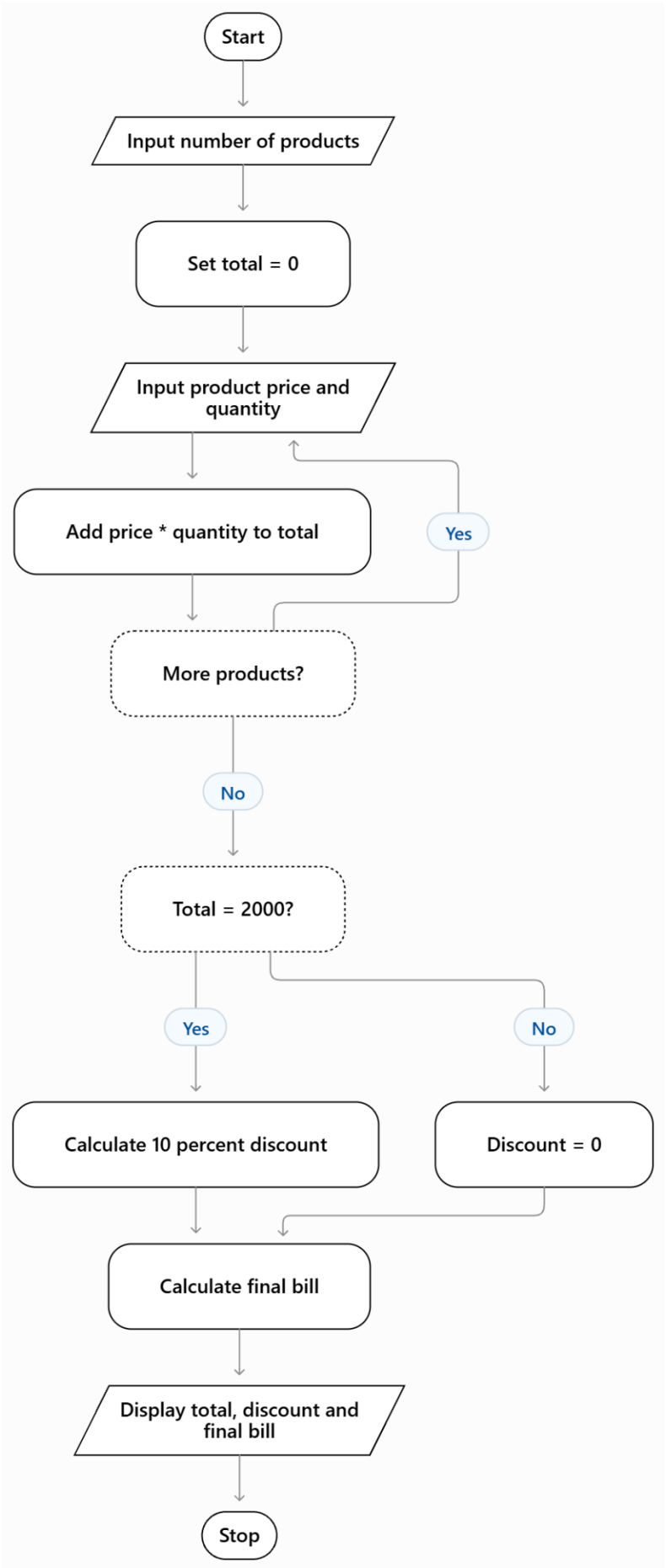
1. Problem Statement

Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2. Algorithm

1. Start.
2. Read the number of products.
3. Initialize total amount to zero.
4. Repeat the following steps for each product:
    - Read product price.
    - Read product quantity.
    - Multiply price by quantity.
    - Add the result to total amount.
5. Check whether total amount is greater than or equal to ₹2,000.
6. If true, calculate a 10% discount.
7. Otherwise, set discount to zero.
8. Calculate final bill by subtracting discount from total amount.
9. Display total amount, discount, and final bill.
10. Stop.

3. Flowchart


Start
Input number of products
Set total = 0
Input product price and quantity
Add price * quantity to total
Yes
More products?
No
Total = 2000?
Yes
No
Calculate 10 percent discount
Discount = 0
Calculate final bill
Display total, discount and final bill
Stop

4. Pseudocode

```
BEGIN

INPUT n

total = 0

FOR i = 0 TO n-1

  INPUT price

  INPUT quantity

  total = total + price * quantity

END FOR

IF total >= 2000 THEN

  discount = total * 0.10

ELSE

  discount = 0

END IF

final_bill = total - discount

PRINT total

PRINT discount

PRINT final_bill
```

STOP

5.Python Program

```
n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))
    total = total + price * quantity

if total >= 2000:
    discount = total * 0.10
else:
    discount = 0

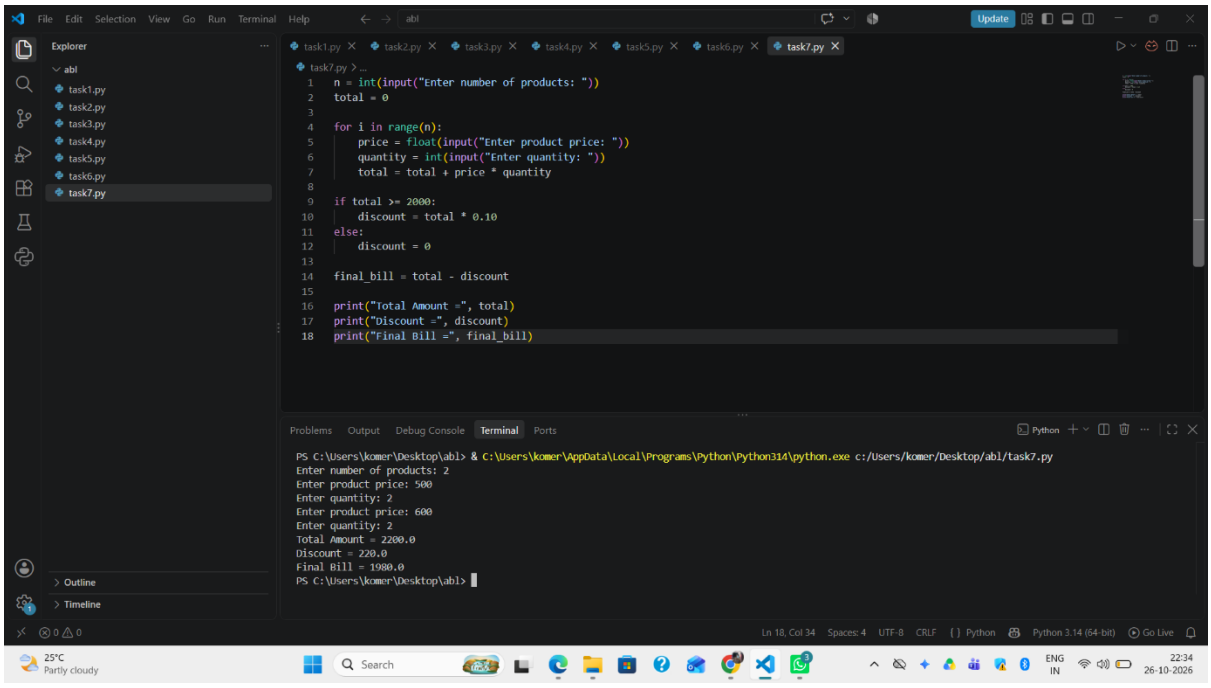
final_bill = total - discount

print("Total Amount =", total)
print("Discount =", discount)
print("Final Bill =", final_bill)
```

6. Test Cases

Test	Total Amount	Discount	Final Bill
1	₹1500	₹0	₹1500
2	₹2000	₹200	₹1800
3	₹3000	₹300	₹2700
4	₹5000	₹500	₹4500
5	₹1000	₹0	₹1000

7.Output Screenshot

1. 

8. Conclusion

The Python program successfully calculates the total shopping bill based on product prices and quantities. It uses a for loop to process multiple products and conditional statements to apply a 10% discount when the total amount is ₹2,000 or more.

Task-8

1. Problem Statement

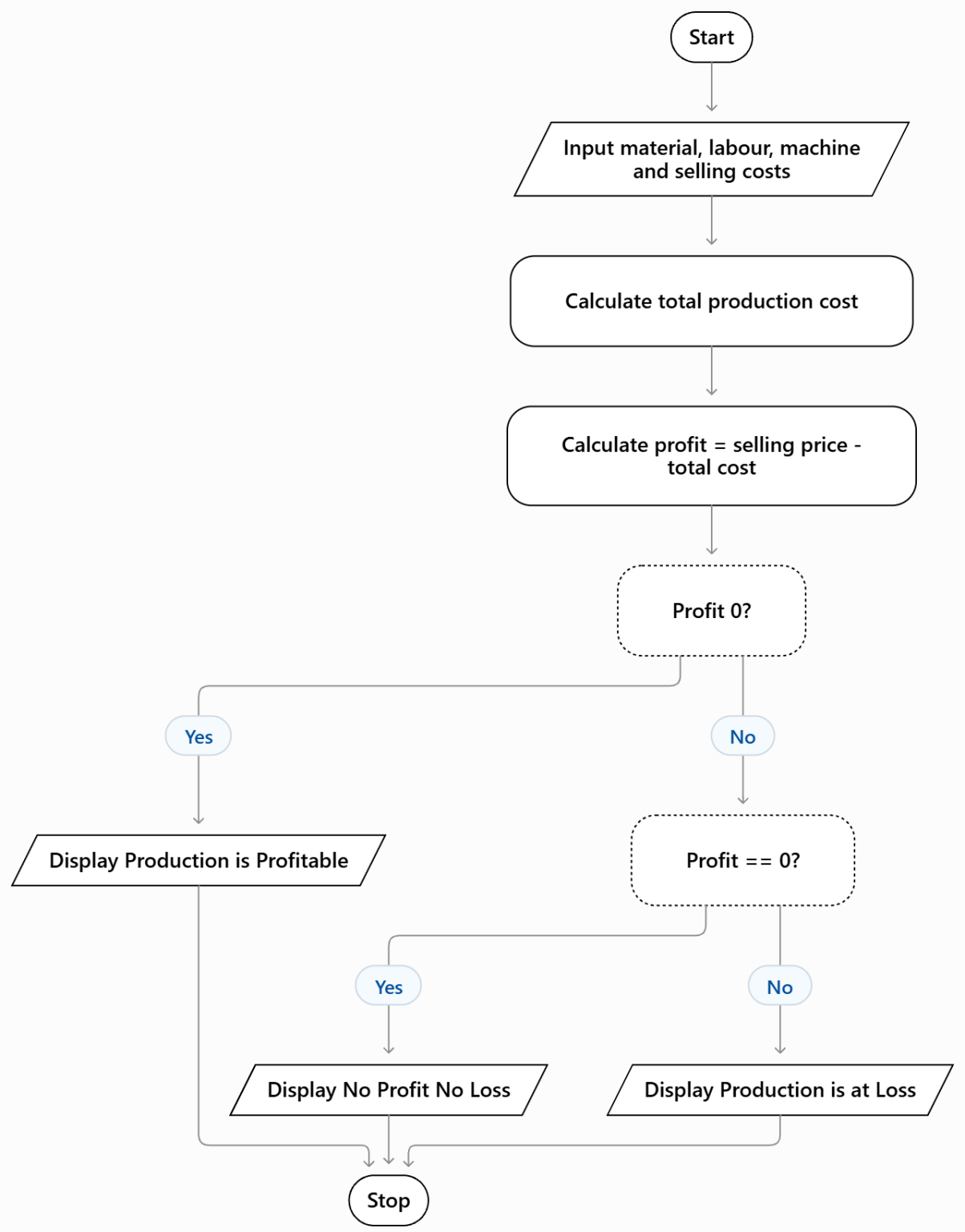
Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. Algorithm

1. Start.
2. Read material cost.
3. Read labour cost.
4. Read machine cost.

5. Read total selling price.
6. Calculate total cost by adding material, labour, and machine costs.
7. Calculate profit by subtracting total cost from selling price.
8. Display total cost and profit.
9. If profit is greater than zero, display "Production is Profitable".
10. Else if profit is equal to zero, display "No Profit No Loss".
11. Otherwise, display "Production is at Loss".
12. Stop.

3. Flowchart



4. Pseudocode

BEGIN

INPUT material

INPUT labour

INPUT machine

INPUT selling

cost = material + labour + machine

profit = selling - cost

PRINT cost

PRINT profit

IF profit > 0 THEN

  PRINT "Production is Profitable"

ELSE IF profit == 0 THEN

  PRINT "No Profit No Loss"

ELSE

  PRINT "Production is at Loss"

END IF

STOP

5.Python Program

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
machine = float(input("Enter machine cost: "))
```

```
selling = float(input("Enter total selling price: "))

cost = material + labour + machine
profit = selling - cost

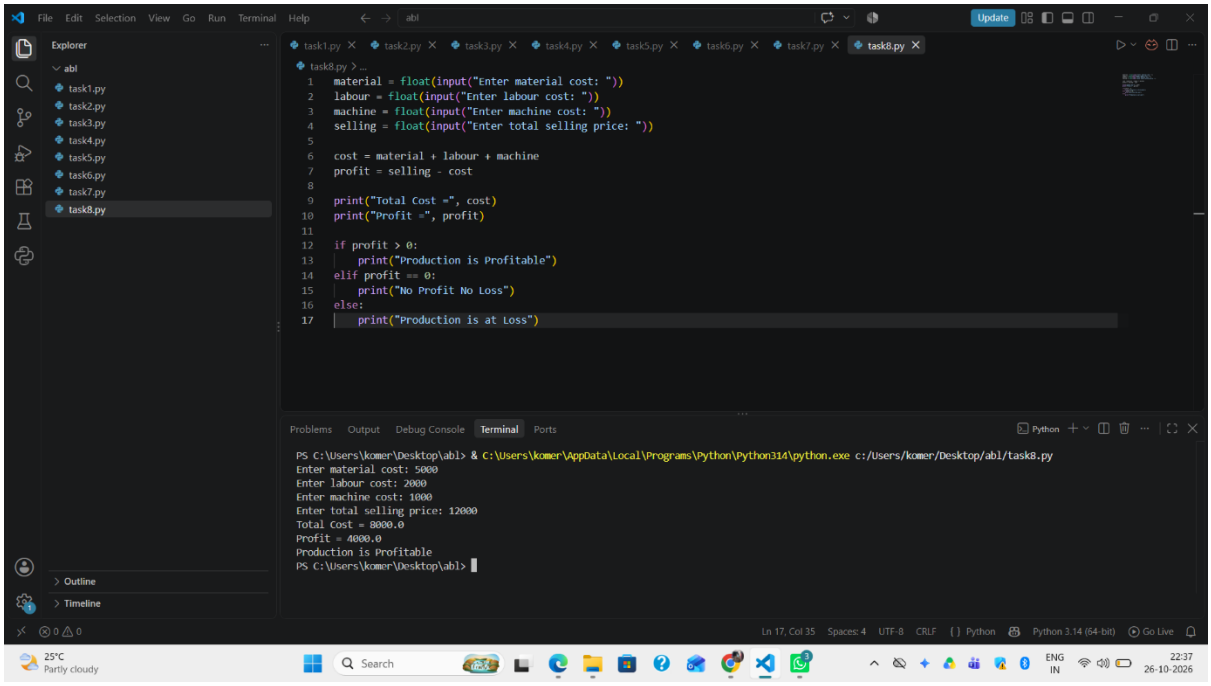
print("Total Cost =", cost)
print("Profit =", profit)

if profit > 0:
    print("Production is Profitable")
elif profit == 0:
    print("No Profit No Loss")
else:
    print("Production is at Loss")
```

6. Test Cases

Test	Material	Labour	Machine	Selling Price	Expected Result
1	1000	500	500	3000	Profit ₹1000
2	1000	500	500	2000	No Profit No Loss
3	1000	500	500	1500	Loss ₹500
4	2000	1000	500	4000	Profit ₹500
5	500	300	200	800	No Profit No Loss

7. Output Screenshot



8. Conclusion

The Python program successfully calculates the total production cost and profit or loss using arithmetic operations and conditional statements. It identifies whether the production process is profitable, has no profit and no loss, or results in a loss.

Task-9

1. Problem Statement

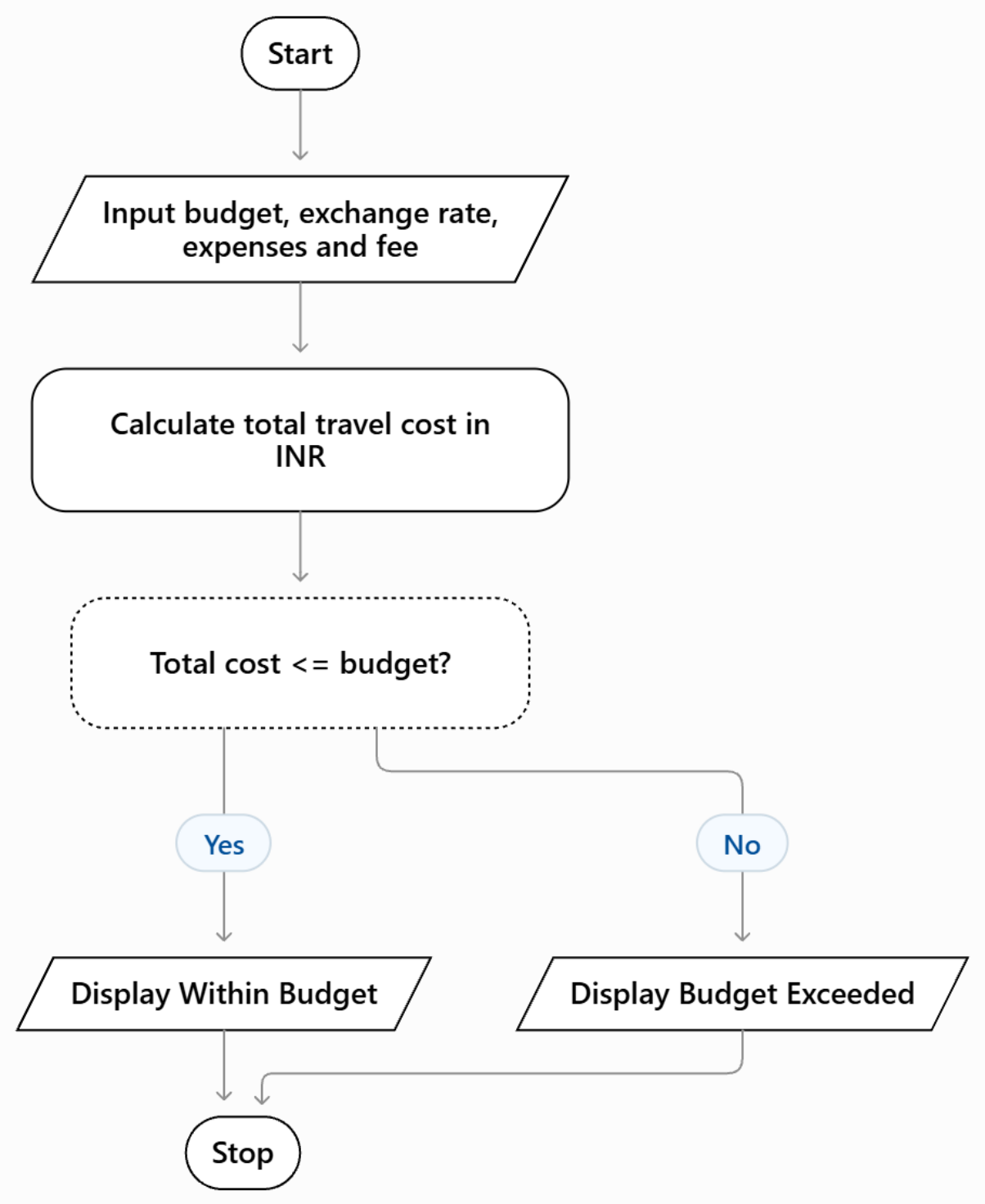
Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

2. Algorithm

1. Start.
2. Read budget in INR.
3. Read exchange rate.
4. Read foreign currency expenses.
5. Read conversion fee.

6. Calculate total travel cost by multiplying foreign expenses by the exchange rate and adding the conversion fee.
7. Display total travel cost in INR.
8. Check whether total cost is less than or equal to the budget.
9. If true, display "Within Budget".
10. Otherwise, display "Budget Exceeded".
11. Stop.

3. Flowchart



6. Pseudocode

BEGIN

INPUT budget

```
INPUT rate

INPUT expense

INPUT fee


total = expense * rate + fee


PRINT total


IF total <= budget THEN

   PRINT "Within Budget"

ELSE

   PRINT "Budget Exceeded"

END IF


STOP
```

4. Python Program

```
budget = float(input("Enter budget in INR: "))

rate = float(input("Enter exchange rate (INR per foreign unit): "))

expense = float(input("Enter foreign currency expenses: "))

fee = float(input("Enter conversion fee in INR: "))


total = expense * rate + fee


print("Total Travel Cost in INR =", total)


if total <= budget:

   print("Within Budget")
```

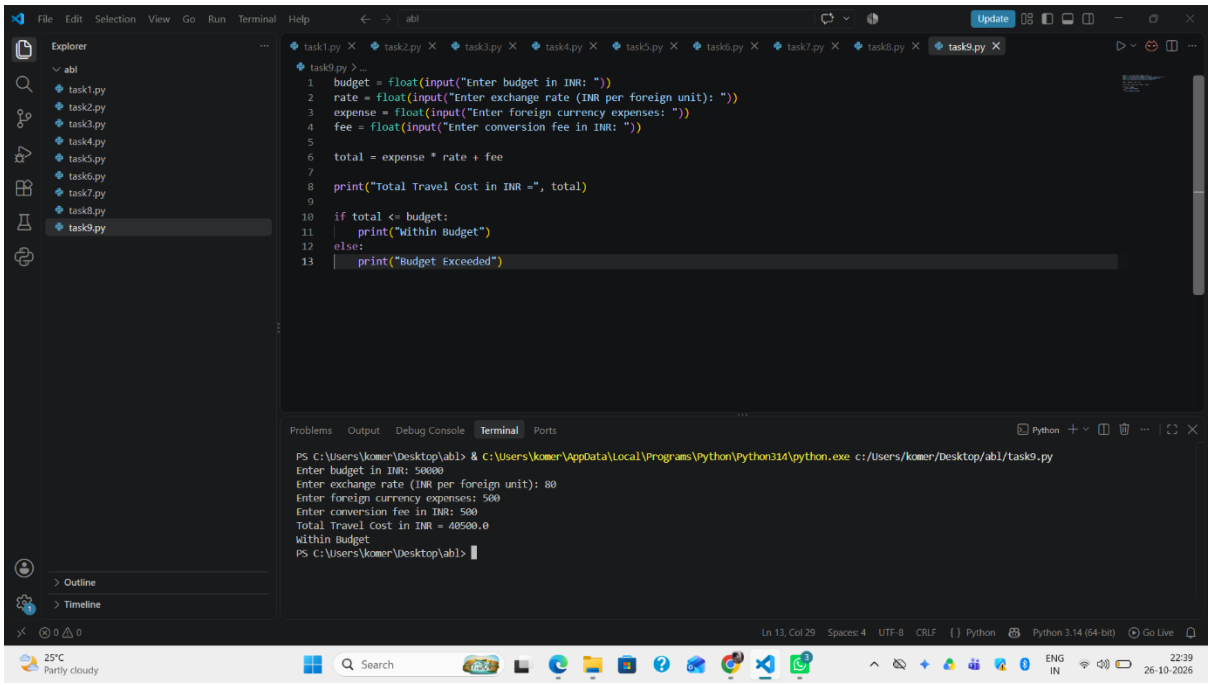
else:

print("Budget Exceeded")

5. Test Cases

Test	Budget	Rate	Expense	Fee	Expected Result
1	50000	85	500	500	Within Budget
2	30000	85	400	500	Budget Exceeded
3	60000	90	500	1000	Within Budget
4	20000	80	300	500	Budget Exceeded
5	50000	80	500	0	Within Budget

7. Output Screenshot



8. Conclusion

The Python program successfully calculates international travel expenses in Indian Rupees using the exchange rate and conversion fee. It uses arithmetic operations and conditional statements to check whether the total travel cost is within the available budget or exceeds it.

Task-10

1. Problem Statement

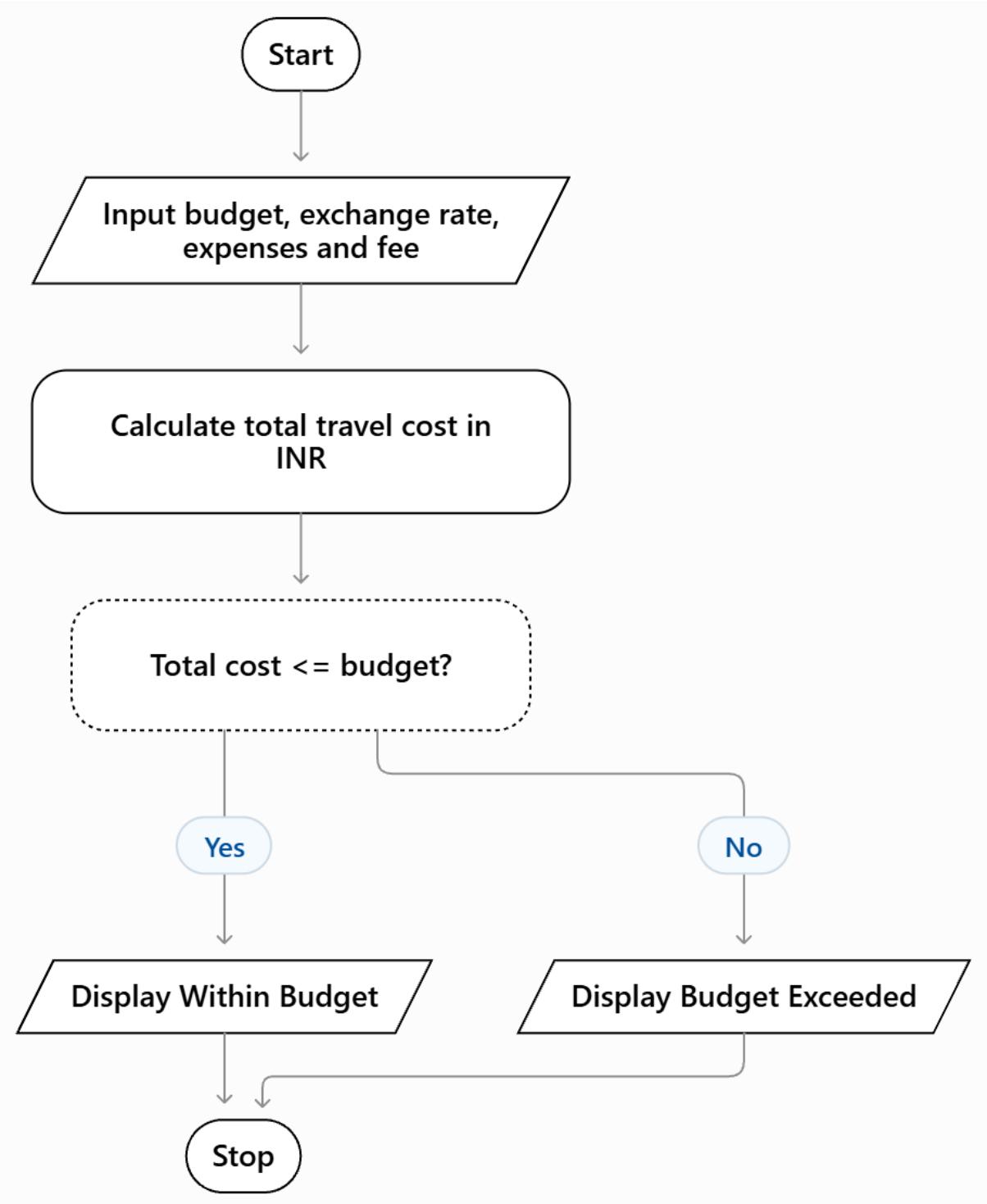
Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

2. Algorithm

1. Start.
2. Read venue cost, equipment cost, catering cost, transport cost, and certificate cost.
3. Read sponsorship amount and contribution per participant.
4. Calculate total event cost.
5. Calculate remaining cost after sponsorship.
6. If remaining cost is less than or equal to zero, display that sponsorship fully covers the event cost.
7. Otherwise, check whether the contribution is greater than zero.
8. Calculate the minimum number of participants required.
9. Display the result.
10. Stop.

3. Flowchart

4.



Pseudocode

START

INPUT venue, equipment, food, transport, certificates

INPUT sponsorship, contribution

```
total_cost = venue + equipment + food + transport + certificates

remaining = total_cost - sponsorship

DISPLAY total_cost
DISPLAY sponsorship

IF remaining <= 0 THEN
   DISPLAY "Event cost fully covered by sponsorship"

ELSE IF contribution > 0 THEN
   participants = INTEGER(remaining / contribution)

   IF remaining MOD contribution != 0 THEN
      participants = participants + 1
   END IF

   DISPLAY participants

ELSE
   DISPLAY "Participant contribution must be positive"
END IF

STOP
```

5. Python Program (Complete Code)

```
venue = float(input("Enter venue cost: "))
```

```
equipment = float(input("Enter equipment cost: "))
food = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificates = float(input("Enter certificate cost: "))

sponsorship = float(input("Enter sponsorship amount: "))
contribution = float(input("Enter fee per participant: "))

total_cost = venue + equipment + food + transport + certificates
remaining = total_cost - sponsorship

print("Total Event Cost =", total_cost)
print("Sponsorship =", sponsorship)

if remaining <= 0:
   print("Event cost fully covered by sponsorship")
elif contribution > 0:
   participants = int(remaining / contribution)

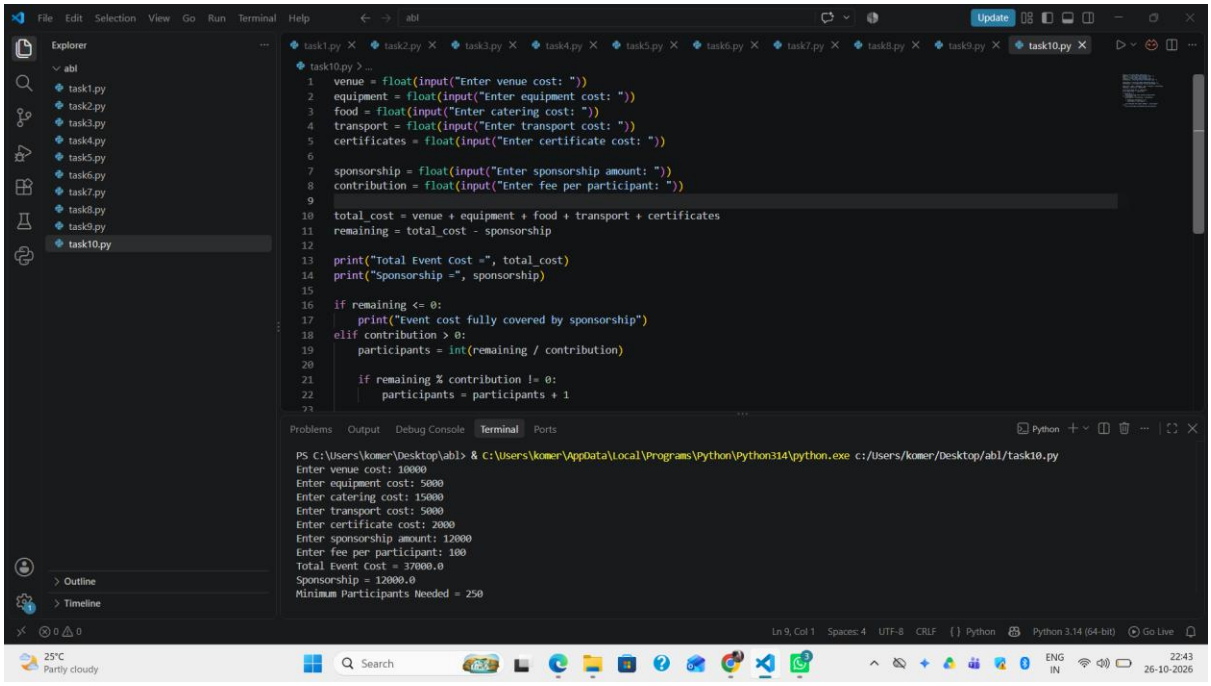
   if remaining % contribution != 0:
      participants = participants + 1

   print("Minimum Participants Needed =", participants)
else:
   print("Participant contribution must be positive")
```

6. Test Cases

Test	Total cost	Sponsorship	Contribution	Expected result
1	₹50,000	₹10,000	₹500	80 participants
2	₹40,000	₹40,000	₹500	Fully covered
3	₹30,000	₹5,000	₹1,000	25 participants
4	₹20,000	₹5,000	₹0	Contribution must be positive
5	₹10,000	₹2,000	₹300	27 participants

7. Output Screenshot



8. Conclusion

The Event Budget and Participant Contribution Calculation System successfully calculates the total event cost, sponsorship amount, and minimum number of participants required to cover the remaining expenses. It helps in planning an event budget efficiently using Python.